

Modelo cliente-servidor

El modelo cliente-servidor es un patrón de arquitectura de software ampliamente utilizado en el desarrollo de aplicaciones y sistemas distribuidos.

Divide una aplicación en dos partes principales:

- Cliente: Es la interfaz de usuario o aplicación que el usuario final utiliza. Solicita servicios o recursos al servidor.
- Servidor: Es un programa que se ejecuta continuamente en un equipo, esperando y respondiendo a las solicitudes de los clientes.

Características clave:

- Comunicación: Los clientes y servidores se comunican a través de una red, generalmente usando protocolos estándar como HTTP.
- Distribución de tareas: El servidor maneja el procesamiento pesado y el almacenamiento de datos, mientras que el cliente se enfoca en la interfaz y la interacción del usuario.

Client-Server Model



El protocolo HTTP

- Inicialmente diseñado para transferir documentos de hipertexto (HTML).
- Surgió cuando internet era ARPANET y mucho más pequeña que en la actualidad.
- Al principio, debido a las bajas velocidades de conexión, solo se transfería texto plano.
- Con el tiempo y el aumento de la velocidad de la red, el protocolo evolucionó para permitir la transferencia de otros tipos de datos como imágenes, sonidos, video y voz.
- Esta evolución respondió a las nuevas necesidades que surgieron con el desarrollo de internet.
- Es un protocolo que no guarda estado. Esto significa que *no se mantiene constantemente una conexión el cliente y el servidor*

Pedidos HTTP

Los clientes realizan pedidos mediante el protocolo HTTP a servidores web. Estos pedidos pueden ser de varios tipos (también son llamados “métodos”).

▼ TIPO DE PEDIDO	▼ DESCRIPCIÓN
OPTIONS	Se usa para preguntarle al servidor las diferentes formas de comunicación que soporta.
GET	Pide un recurso al servidor.
HEAD	Igual que GET, pero el servidor sólo devuelve el encabezado de lo pedido.
POST	Método que se usa para enviar información al servidor.
PUT	Usado para enviar recursos al servidor.
DELETE	Borra recursos del servidor.
TRACE	Se usa para pedir un rastreo del pedido.
CONNECT	Método reservado.



Tomcat

Tomcat es un servidor web y contenedor de servlets de código abierto desarrollado por la Apache Software Foundation.

Características principales de Tomcat:

Servidor web: Puede servir páginas web estáticas.

Contenedor de servlets: Ejecuta aplicaciones Java del lado del servidor.

Soporte para JavaServer Pages (JSP): Permite la creación de páginas web dinámicas.

Implementación de especificaciones Java EE: Soporta tecnologías como Servlets, JSP, WebSocket, y más.

Ligero y flexible: Adecuado para entornos de desarrollo y producción.

Configuración personalizable: Mediante archivos XML y propiedades.

Usos comunes:

- Despliegue de aplicaciones web Java
- Entorno de desarrollo para aplicaciones web
- Servidor integrado en IDEs como Eclipse o IntelliJ IDEA

Servlet

Definición: Un Servlet es una clase Java que maneja solicitudes y respuestas en un servidor web. Funciona en el lado del servidor, sin una interfaz de usuario o soporte de HTML por sí mismo, y es utilizado para generar contenido dinámico en páginas web.

Manejo de Solicitudes: Los Servlets son utilizados para procesar solicitudes entrantes desde el cliente (por ejemplo, un navegador web), realizar operaciones como consultas a bases de datos o cálculos, y luego generar una respuesta adecuada, que puede ser una página web HTML, una imagen, etc.

Generación de Contenido Dinámico: A diferencia de los documentos HTML estáticos, los Servlets permiten generar contenido web dinámico. Esto significa que el contenido de la página web puede cambiar en función de los parámetros de la solicitud, la interacción del usuario, o datos obtenidos de otras fuentes como bases de datos.

Fundamentales:

Paquete de Importación: Los servlets comúnmente importan clases de los paquetes `javax.servlet` y `javax.servlet.http`, que contienen las interfaces y clases necesarias para escribir servlets.

Declaración: Un servlet extiende de "HttpServlet" que es una clase abstracta proporcionada por Java en el paquete `javax.servlet.http`.

"HttpServlet" proporciona métodos como `doGet`, `doPost`, `doPut`, `doDelete`, entre otros.

Fundamentales:

Métodos Principales:

doGet(): Maneja las solicitudes GET, utilizadas principalmente para solicitar datos del servidor.

doPost(): Maneja las solicitudes POST, utilizadas para enviar datos al servidor, como los formularios web.

Objetos Request y Response: HttpServletRequest y HttpServletResponse son objetos fundamentales que representan la solicitud del cliente y la respuesta del servlet, respectivamente